

Архитектура компьютера и основы ОС Linux

Основы устройства компьютера

Что такое компьютер. Примеры компьютеров. Элементная база. Машинный код. Регистры, прерывания, порядок байт.

Оглавление

[Что такое компьютер](#)

[Компьютер и программа. Машина Тьюринга](#)

[Математическая база](#)

[Элементная база](#)

[4 века компьютеров. Лампы, транзисторы, БИС, СБИС](#)

[Процессор](#)

[История Intel-процессоров](#)

[Разрядность процессора](#)

[Регистры процессора](#)

[Порядок байтов](#)

[Прерывания](#)

[Исключения](#)

[Аппаратные прерывания](#)

[Немаскируемое прерывание](#)

[Прерывание для отладки – breakpoint](#)

[Программные прерывания](#)

[Системные вызовы](#)

[Языки программирования низкого уровня](#)

[Машинный код](#)

[Мнемоники, опкоды, дизассемблер](#)

[Практическое задание](#)

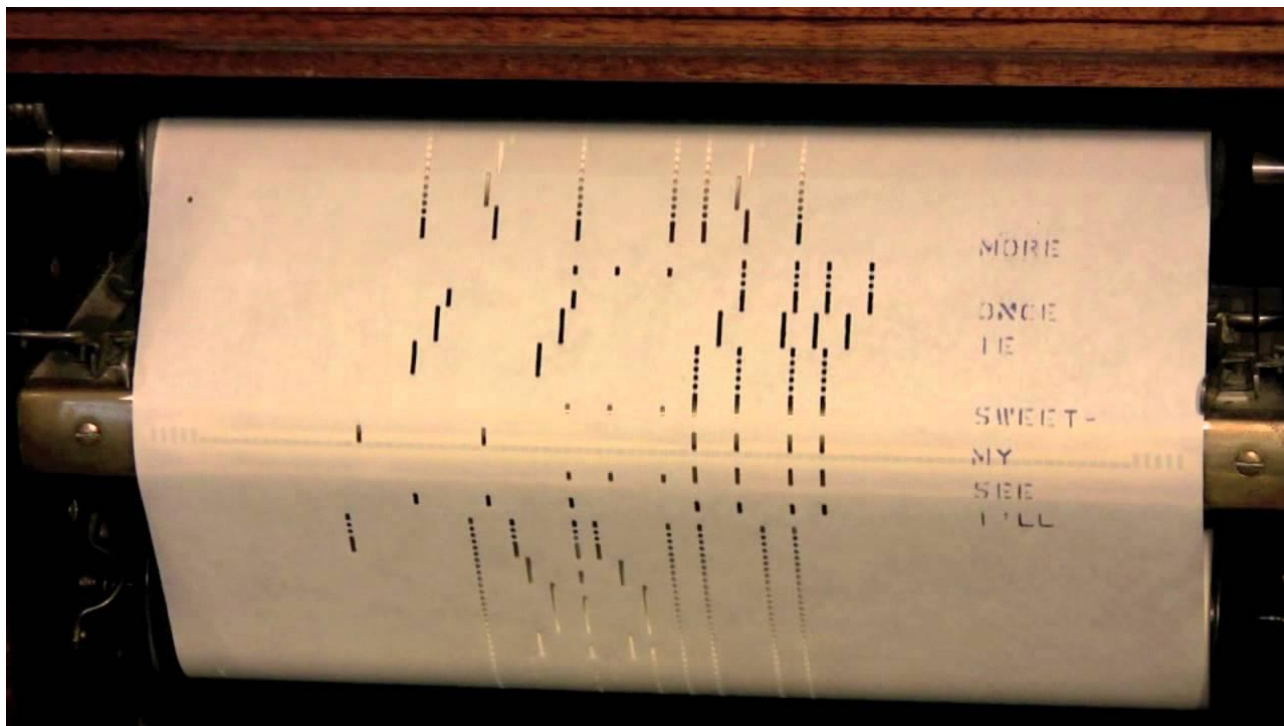
[Дополнительные материалы](#)

[Используемая литература - это обязательный пункт](#)

Что такое компьютер

Компьютер и программа. Машина Тьюринга

Компьютеры появились в связи с необходимостью расчетов, прежде всего для военных нужд. А программы — задолго до вычислительных машин. Механическая пианола, позволяющая воспроизводить музыку с записи, впервые заиграла в 1887 году, а шарманки были известны с XV века.



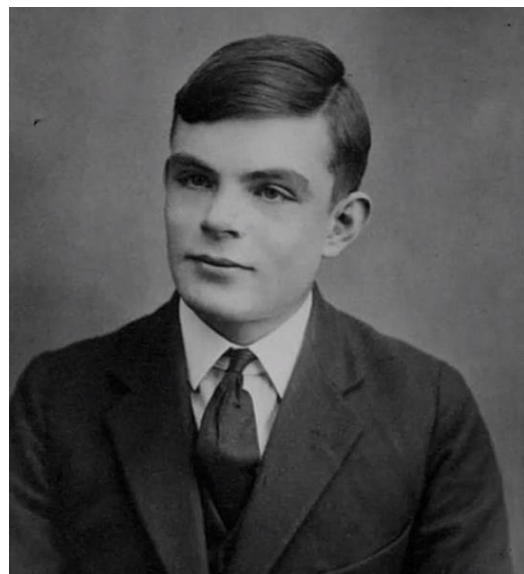
Запись мелодии для пианолы: https://www.youtube.com/watch?v=Z3ekShc_h-w

Но, разумеется, вычислений такие программы производить не могли. В них предусматривалось только последовательное движение вперед без возможности произвольного перехода к другому участку программы, а в качестве машины выступал музыкальный инструмент.

Теперь представим себе, что лента, которую считывает машина, бесконечна. При этом машина может прокручивать ее вперед или назад в зависимости от считанной информации, а также записывать на ленту новые данные. Внимание! Мы «изобрели» машину Тьюринга. Это считывающее устройство, находящееся в одном из нескольких состояний, умеющее читать ленту, изменять состояние в зависимости от прочитанного, перемещаться по ленте вперед и назад в зависимости от состояний, а также выполнять записи. Число состояний, в которых может находиться машина Тьюринга, конечно и заранее определено.

Алан Тьюринг — отец IT-индустрии и концепции Машины Тьюринга

Концепция этой машины была предложена Аланом Тьюрингом в 1936 году для формализации понятия алгоритма. Идея машины Тьюринга тесно связана с понятиями процессора,



алгоритма, языка программирования, транслятора, компилятора, интерпретатора, виртуальной машины (машины, выполняющей байт-код).

Языки программирования называются Тьюринг-полными, если на них можно реализовать любую вычислимую функцию. К примеру, регулярные выражения не являются Тьюринг-полным языком, а C или PHP — являются. Отметим, что языки программирования — это настолько же давний компонент компьютеров, как и операционная система. Первоначально они сами играли роль ОС, а сейчас входят в ее состав в виде командных интерпретаторов, компиляторов для сборки ПО из исходных кодов, библиотек и заголовочных файлов.



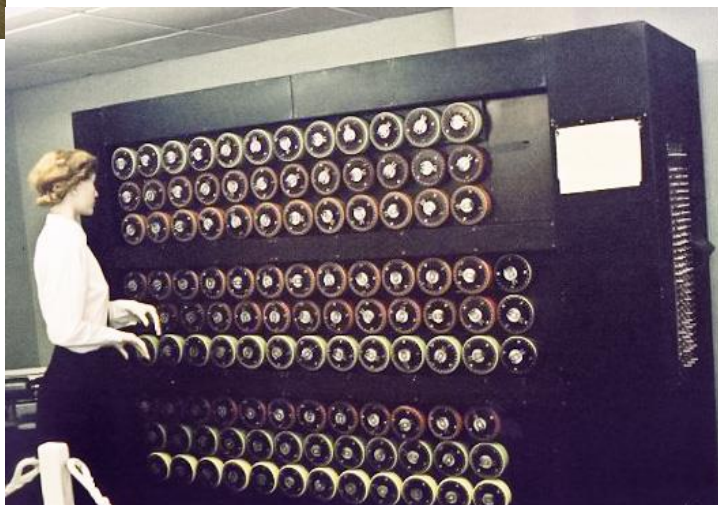
«← Немецкая шифровальная машина «Энигма»

В каком-то смысле тому, что у нас есть компьютеры, мы обязаны немецкой шифровальной машине «Энигма». Главное, что сделало ее предшественницей компьютеров, — способность к преобразованию информации. На вход подается информация в текстовом виде, которую нужно зашифровать; машина производит преобразования, и на выходе получается результат обработки — информация в зашифрованном виде.

Алан Тьюринг фактически занимался реверс-инжинирингом — обратной разработкой, — построив машину, которая, не зная исходных данных, могла дешифровывать информацию, закодированную «Энигмой».

Turing Bombe →

Для дешифровки была создана электронно-механическая машина Turing Bombe (криптологическая бомба Тьюринга). Теоретическую работу выполнил Алан Тьюринг. Над созданием машины трудились другие люди, но Тьюринг вынес из этой работы то, что мы знаем как машину Тьюринга, — этот теоретический аппарат лежит в основе и компьютеров, и процессоров, и даже языков программирования.



Машину Тьюринга мы с вами «придумали». Осталось реализовать вычислитель — электронное устройство, которое может менять состояния. Для этого необходимо найти или разработать элементную базу.



Brunsviga (Брунсвига) мод. М III

Первые счетные устройства были механическими. Арифмометры — еще одни предшественники современных компьютеров. Арифмометр получает на вход числа, выполняет арифметические действия (складывает, умножает, делит и вычитает), и на выходе показывает результат.

Арифмометры стоит воспринимать не как исторический курьез, а как наглядный инструмент. Например, на [видеоролике](#) вы можете наблюдать, почему на ноль делить нельзя.

Разберем, как арифмометр делит 8 на 2. Счетчик содержит 0. Мы вычитаем 2 из 8, получаем 6. Счетчик содержит 1. Мы вычитаем 2 из 6, получаем 4. Счетчик содержит 2. Мы вычитаем 2 из 4. Получаем 2. Счетчик содержит 3. Далее мы вычитаем 2 из 2, получаем ноль и останавливаемся. А счетчик содержит 4 — результат деления 8 на 2.

Пытаемся разделить 8 на 0. Счетчик содержит 0. Вычитаем 0 из 8. Результат 8. Счетчик содержит 1. Вычитаем 0 из 8. Результат 8. Счетчик содержит 2. Вычитаем 0 из 8. Результат 8. Счетчик содержит 3. Машина никогда не остановится. Поэтому современные процессоры не допускают такую операцию — чтобы не было заикливания или переполнения (когда счетчик не сможет вместить нужное число). Такие операции вызывают исключения.



И «Энигма», и «Бомба Тьюринга», и арифмометр не являются машинами общего назначения. Но концепция машины Тьюринга позволяла сформировать такой аппарат, который мог бы исполнять любую программу (не только воспроизводить музыку, но и дешифровать шифрограммы, выполнять арифметические вычисления, и другое — в зависимости от программы).

Меня программы, мы могли бы заставить компьютер решать разные задачи. Предшественниками программ были как раз перфорированные листы бумаги для записи музыки или карты для ткацких машин Жаккарда.

Оставалось научить машину «принимать решения» в зависимости от того, что указывает ей программа. И для этого необходимо собрать нужную элементную базу.

Математическая база

Но прежде чем перейти к элементной базе, нужно коснуться математической. Чтобы элементы могли выполнять действия, нужно определить, какие операции будут выполняться и над какими операндами.

Практически вся современная техника работает в двоичной системе счисления (хотя были эксперименты по созданию троичных компьютеров — ЭВМ «Сетунь» Н. П. Брусенцова, — а также в связи используется более двух уровней сигнала). Низкое напряжение или его отсутствие обозначает 0, повышенное — 1. Мы используем систему счисления по основанию 10, то есть цифры от 0 до 9. Но если бы у людей было не 10, а 8 пальцев, то цифры использовались бы от 0 до 7, а число 10 означало наше 8, 11 — наше 9. Так как у машины всего два «пальца», то есть два состояния, используются две цифры — 0 и 1.

Соответственно, 1 — это 1, 10 — это наше 2, 11 — это наше 3, 100 — наше 4, 101 — наше, «десятичное», пять. С двоичной системой работать сложно, так как она не кратна 10. Поэтому для удобства стали использовать системы счисления, кратные двум. Изначально по основанию 8 (цифры от 0 до 7), а в дальнейшем и по основанию 16 (цифры от 0 до F, то есть к 10 привычным числам добавили числа от A до F, которые при переводе в десятичную систему счисления означают диапазон от 10 до 15. 16 уже кодируется как 10 в шестнадцатеричной системе счисления).

Рассмотрим числа в разных системах счисления

Десятичная	Двоичная	Двоичная (с выравниванием)	Восьмеричная	Шестнадцатеричная	Шестнадцатеричная (с выравниванием)
0	0	0000	0	0	00
1	1	0001	1	1	01
2	10	0010	2	2	02
3	11	0011	3	3	03
4	100	0100	4	4	04
5	101	0101	5	5	05
6	110	0110	6	6	06
7	111	0111	7	7	07
8	1000	1000	10	8	08
9	1001	1001	11	9	09
10	1010	1010	12	A	0A
11	1011	1011	13	B	0B
12	1100	1100	14	C	0C
13	1101	1101	15	D	0D
14	1110	1110	16	E	0E
15	1111	1111	17	F	0F
16	10000	0001 0000	100	10	10
17	10001	0001 0001	101	11	11
18	10010	0001 0010	102	12	12
.....					
254	11111110	1111 1110	366	FE	FE
255	11111111	1111 1111	377	FF	FF
256	100000000	0001 0000 0000	400	100	0100

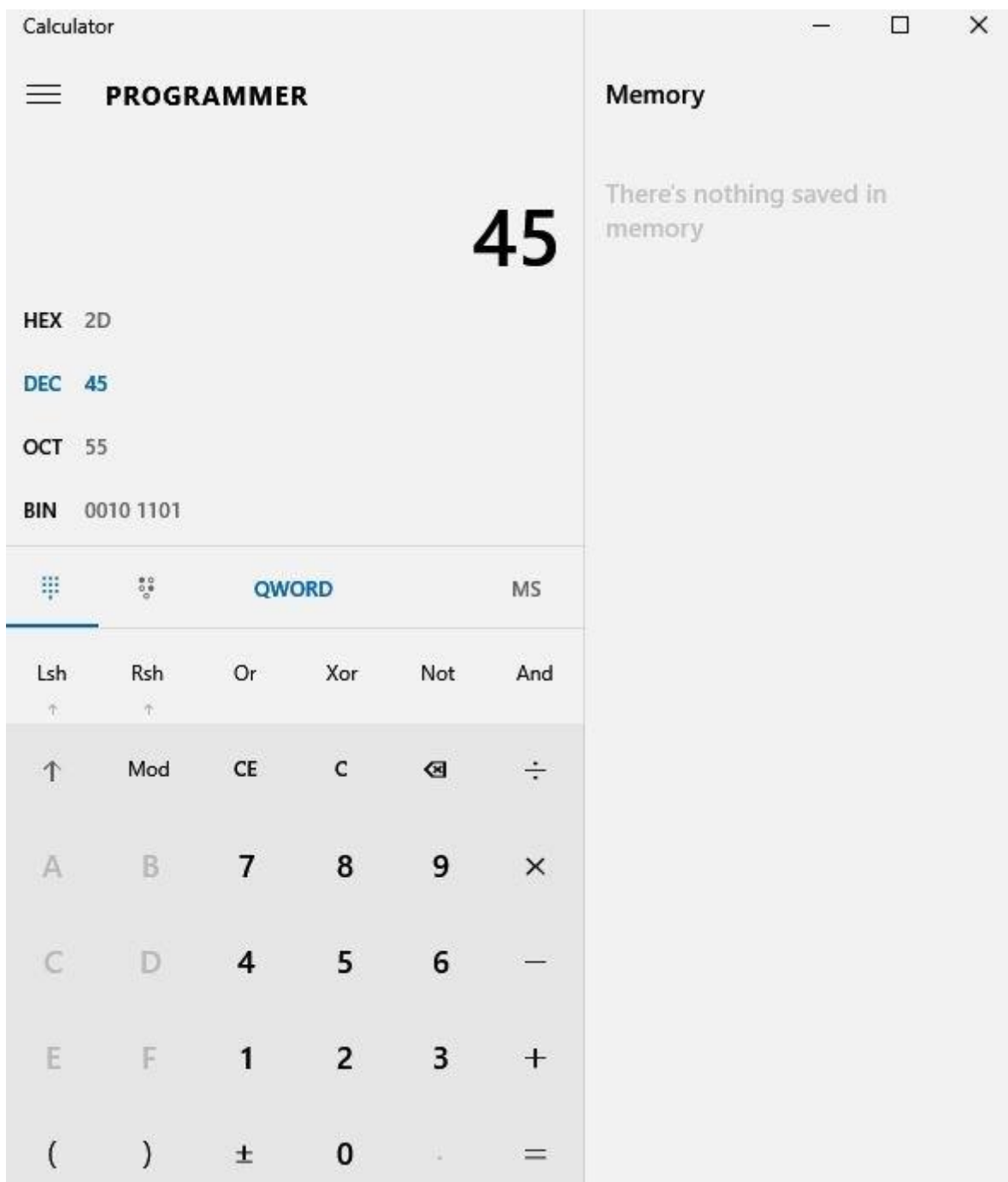
Если вы впервые встречаетесь с позиционными системами счисления (а описанные выше к ним и относятся), то рекомендуем поэкспериментировать с калькулятором. Например, встроенный CALC в Windows 10 в режиме Programmer позволяет видеть, как числа выглядят в разных системах счисления, и переводить их из одной в другую.

Выбираете режим, в котором вы вводите:

- HEX — шестнадцатеричная (hexadecimal);
- DEC — десятичная (decimal);

- OCT — восьмеричная (octal);
- BIN — двоичная (binary).

В остальных системах счисления вы сразу видите результат.



Над двоичными числами можно выполнять те же операции, что и над десятичными: арифметические и битовые (логические и сдвиги).

Арифметические операции похожи на привычные нам:

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 10$$

Можно складывать в столбик:

$$10 +$$

$$10$$

$$=$$

$$100$$

или

$$10 +$$

$$11$$

$$=$$

$$101$$

Но есть и логические операции. В данном случае мы говорим о битовых логических операциях, так как при работе с байтами мы производим операцию над каждым соответствующим разрядом.

Двоичные 0 и 1 используются для обозначения логических состояний, при этом 0 обозначает, как правило, ложь (false), а 1 — истину (true).

Рассмотрим битовые логические операции:

- И (AND), также обозначается как &, а в математике $\wedge$;
- ИЛИ (OR), также обозначается как |, а в математике $\vee$;
- ИЛИ (XOR), также обозначается как ^, а в математике $\oplus$ (еще эта операция называется сложением по модулю 2).

Это были бинарные операции.

Есть и унарная операция отрицания НЕ (NOT) — обозначается как ! или ~, а в математике $\neg$ ($\neg a$) либо надчеркиванием $\bar{a}$.

Для работы с логическими битовыми операциями используют таблицы истинности. Они похожи на таблицу умножения, но состоят из перебора значений переменных и результата выполнения операции над ними.

Самая простая таблица истинности у операции отрицания.

a	$\sim a$
0	1
1	0

Таблица истинности для логического И:

a	b	$a \& b$
0	0	0
0	1	0
1	0	0
1	1	1

Таблица истинности для логического ИЛИ:

a	b	$a b$
0	0	0
0	1	1
1	0	1
1	1	1

Таблица истинности для исключающего ИЛИ (XOR):

a	b	$a \wedge b$
0	0	0
0	1	1
1	0	1
1	1	0

Теперь, подавая сигналы на элементы, мы сможем добиваться операций, — если будем считать, что у нас есть входы и выходы на соответствующие схемотехнические элементы, которые умеют выполнять операции И, ИЛИ, отрицания и более сложные, такие как складывание или сдвиги.

Пример битового сдвига влево: было 0000 0010 — стало 0000 0100. Фактически это очень быстрое умножение на 2.

Но нам надо научиться не только вычислять, но и управлять событиями. Для этого нужны элементы, которые могут принимать управляющие сигналы, и в зависимости от поступившего сигнала (тех же 0 или 1) производить необходимые действия.

Элементная база

Реле — простейшее устройство, которое можно использовать для управления. Relay означает «связь», и сначала реле применяли в телеграфе для усиления сигнала.

Первые вычислительные машины создавались на базе электромеханических реле. Это не удивительно, поскольку реле — простейшее устройство, позволяющее изменять состояния. Так, одна из первых ЭВМ — машина Z3, разработанная Конрадом Цузе в 1941 году, — действовала на основе телефонных реле. В том же году появилась и американская машина MARK-1, в основе которой также были электромеханические реле и переключатели. В 1944 году Конрад Цузе уже работал над машиной Z4, одновременно создавая один из первых языков программирования высокого уровня — Планкалькуль.

Конечно, были эксперименты по построению вычислительных машин на других принципах. Например, в 1936 году Владимир Лукьянов создал гидравлический интегратор — единственную в СССР вычислительную машину для решения дифференциальных уравнений в частных производных. Но это была аналоговая машина, ориентированная на решение неуниверсальных задач. Для построения вычислительных машин, работающих с состояниями, именно реле на тот момент было оптимальным электрическим переключателем.

Электромеханическое реле — это простейший прибор с электромагнитом, позволяющий замыкать, размыкать или даже переключать линию. Чтобы механический переключатель сработал, на управляющий контакт подается ток.

На рисунке — пример реле-включателя.

Такое реле похоже на обычный выключатель, наподобие тех, которыми мы включаем и выключаем настольные лампы. Но включение-выключение производится подачей или прекращением подачи тока (соответственно, поступлением сигнала 1 или сигнала 0).

Подается ток, магнит притягивает сердечник, в результате два управляющих контакта замыкаются.

Но реле может не только включать и выключать, но еще и переключать. Если подвести два провода и соединить примагничиваемый сердечник к третьему контакту-якорю, то при подаче напряжения будут замыкаться 2 и 3, а при его отсутствии — 1 и 2.

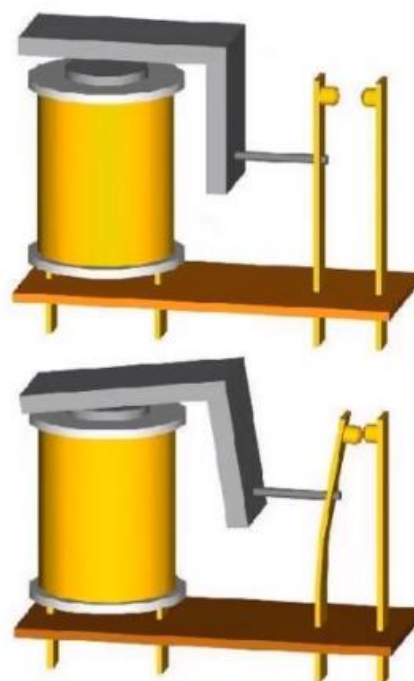
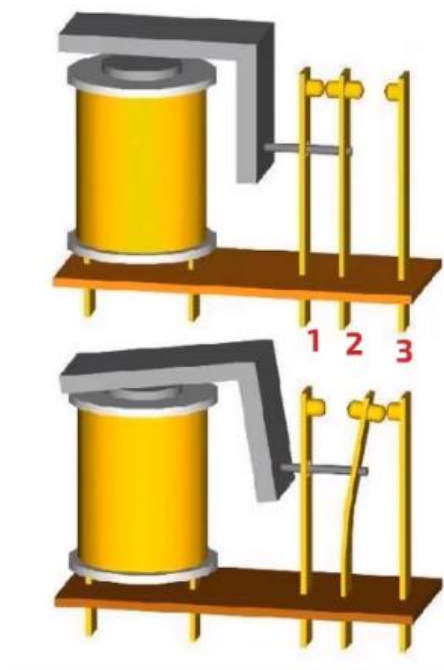


Схема работы электромагнитного реле:



1 — электромагнит;

2 — подвижный якорь;

3 — переключатель.

Разумеется, подобный способ управления позволяет задавать определенную логику работы за счет комбинирования переключателей. Недостатки такого подхода — громоздкость схемы, медленный темп работы из-за механического движения якоря и переключателя, а также ненадежность. Есть легенда, что известный термин *bug*, означающий программную ошибку, возник из-за жука, который застрял в реле и тем самым вывел переключатель из строя. В любом случае подвижные переключатели являются недостаточно надежными элементами для вычислительных устройств, так как быстро изнашиваются, теряют подвижность и могут быть замкнуты посторонними предметами.

Для дальнейшего развития вычислительной техники требовалась более совершенная и надежная элементная база — и ею стали вакуумные лампы, или радиолампы.

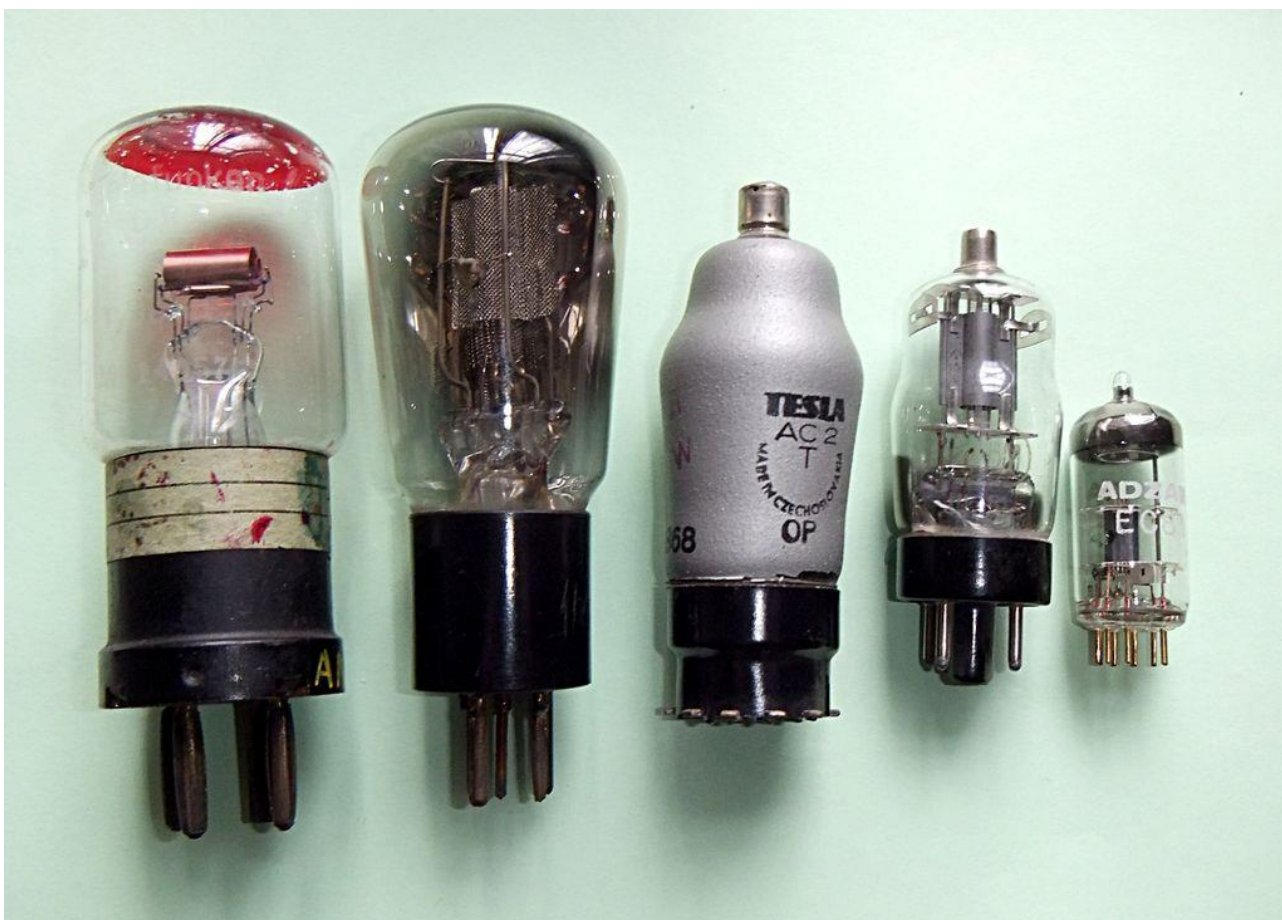
Четыре века компьютеров: лампы, транзисторы, БИС, СБИС

Что помогло лампам заслужить такую любовь пользователей — устойчивость к ЭМИ, теплый ламповый звук или красивый внешний вид? В свое время вакуумные лампы, или радиолампы, произвели революцию в электротехнике, позволив создать более совершенные вычислительные машины.

Эффект, используемый в вакуумных лампах, был открыт Томасом Эдисоном в 1881 году, и это стало мощным стимулом к дальнейшему развитию электронных ламп. Среди нововведений того времени важно отметить вакуумный диод, в котором лампа пропускает ток только в одном направлении, и триод. Имея дополнительный вход, эта лампа позволяла управлять током.



Вакуумные диоды. By RJB1 - Own work, GFDL, <https://commons.wikimedia.org/w/index.php?curid=14549203>



Вакуумные триоды — By RJB1 — Own work, GFDL, <https://commons.wikimedia.org/w/index.php?curid=14533720>

Первые вычислительные машины, работающие на лампах, были построены в 1943 году. Это были британский «Колосс» и американский ЭНИАК — предок не менее известного ЭДСАКА, созданного уже в 1948 году.



ЭНИАК

Устройства первого поколения, работавшие на радиолампах, были полноценными программируемыми машинами: они обладали процессором, оперативной памятью, позволяли довольно быстро выполнять вычисления. Но и программирование, и ввод программ требовали ручную работу в машинном коде (даже ассемблера тогда не было) путем переключения соответствующих тумблеров (как на фото). Ни о языках программирования, ни об операционных системах речь еще не шла.

Если основным недостатком машин на электромагнитных реле была откровенная ненадежность, машины на радиолампах, помимо этого обладали еще и внушительными размерами.

В 1947 году физики Уолтер Браттейн и Джон Бардин разработали первый работоспособный точечный транзистор — полупроводниковый прибор, работающий на иных физических принципах, чем радиолампа, но позволяющий выполнять те же задачи. В 1948–1950 годах Уильям Шокли создал теорию p-n-перехода и плоскостного транзистора, а уже в 1954 году Texas Instruments выпустила первый кремниевый транзистор.

Транзисторы оказались более надежными, чем лампы, слабее нагревались и потребляли меньше электроэнергии. Это дало новые возможности для развития вычислительной техники.



Первый транзистор

В машинах второго поколения стали применяться магнитные диски, а для ввода-вывода информации использовались перфоленты и перфокарты. Активно развивается программное обеспечение: появляются библиотеки программ и такие программы, как автокод и монитор. Разрабатываются языки программирования (Фортран, Алгол), трансляторы (интерпретаторы и компиляторы). К наиболее известным машинам второго поколения относятся UNIVAC и «Минск-22».

Для машины IBM-704 в Bell Labs (подразделение AT&T) была создана операционная система BESYS. Она предназначалась для эффективного выполнения большого количества коротких задач, динамически загружаемых с использованием перфокарт (для этого использовалось дополнительное оборудование для работы с перфокартами и печати результатов на бумаге). Такой подход послужил прообразом для операционных систем с разделяемым временем исполнения задач.

Интересным экземпляром машин второго поколения, разработанных в СССР, была «Сетунь», работавшая на троичной логике (использовались триты и трайты вместо битов и байтов). «Сетунь» была создана под руководством Н. П. Брусенцова в 1958 году. С используемым в ней машинным кодом и особенностями его применения можно ознакомиться [в этой статье](#).

В 1958–1959 годах был изобретен способ изолировать компоненты на кристалле полупроводника — таким образом в одном корпусе мог помещаться не один транзистор, а целая схема, что вело к миниатюризации устройств. Так начался век микросхем.

Под микросхемой понимается интегральная схема — кристалл или пленка со схемой, — заключенная в корпус. На интегральных схемах (ИС) были реализованы еще более быстрые и меньшие по размеру ЭВМ третьего поколения. Во второй половине 60-х годов в США компания IBM начала выпускать серии машин IBM-360, затем IBM-370, и, наконец, в 70-х годах в СССР стали создавать вычислительные машины серии ЕС ЭВМ (Единая система ЭВМ) по образцу IBM 360/370.



IBM/360, еще без Intel Inside

*Автор: Ben Franske — DM IBM S360.jpg on en.wiki, CC BY 2.5,
<https://commons.wikimedia.org/w/index.php?curid=1189162>*

Возможности BESYS для компьютеров третьего поколения уже не удовлетворяли пользователей, поэтому в 1964 году началась разработка новой операционной системы Multics, которой занимались Массачусетский технологический институт и компании General Electric (GE) и Bell Labs. Это уже была полноценная операционная система с разделением времени выполнения программ. Multics была написана на языке высокого уровня PL/I и работала на 36-битных вычислительных машинах GE-600. В системе Multics был реализован ряд идей, которые сейчас применяются в большинстве операционных систем: среди них работа с файлами на уровне операционной системы, использование виртуальной памяти, позволяющей отображать файлы в сегменты памяти, динамическое связывание программ и библиотек, иерархическая файловая система с именами файлов произвольной длины, несколько имен у файлов, символические ссылки на директории. Кроме того, система Multics позволяла на ходу подключать и отключать оборудование, реализуя модульный принцип. Впрочем, хотя Multics и вошла в число первых ОС, написанных на языке высокого уровня, истинное первенство принадлежало MCP, написанной на Алголе.

AT&T счел Multics коммерчески неуспешной, но группа создателей системы продолжила ее развивать. Новая операционная система, разработанная ими, получила название Unics — производное от Multics: если Multics расшифровывалось как MULTIplexed Information and Computing Service, то Unics — как UNIpIplexed Information and Computing System, что подчеркивало простоту новой системы.

К 1969 году Unics была переписана для использования на мини-компьютерах типа PDP-7. Это была первая редакция системы (Edition I) — ее первая официально выпущенная версия. С 1 января 1970 года компьютеры с UNIX — так теперь называлась новая операционная система — уже отсчитывали системное время. Началась эпоха UNIX.

Первая редакция UNIX, как и многие (но не все) операционные системы того времени, была написана на ассемблере, не содержала ОС и встроенного компилятора с языка высокого уровня. В 1966 году был разработан интерпретируемый язык BCPL, а в 1969 — язык B (Би), также интерпретируемый. По

сути, это была упрощенная для реализации на мини-компьютерах версия BCPL (который, кстати, в свою очередь сам был развитием языка CPL). Наконец, в 1972 году UNIX была переписана на языке В. Это была вторая редакция (Edition II) Unix.



Мини-компьютер PDP-7

By en:User:Toresbe - From english Wikipedia. Original description was: The Oslo PDP-7, before restoration started. I took the picture., CC SA 1.0, <https://commons.wikimedia.org/w/index.php?curid=1963657>

В 1969–1973 годах на основе Би был разработан компилируемый язык, получивший название С (Си). На языке Си и была написана новая версия UNIX, известная как третья редакция — она включала встроенный компилятор С. В том же году вышла и четвертая редакция. Теперь на Си было переписано и системное ядро (в духе системы Multics, также написанной на языке высокого уровня ПЛ/I). В 1975 году вышла пятая редакция, полностью переписанная на Си. В то время UNIX широко распространяется в учебной и университетской среде, у нее появляются новые версии, разработанные другими организациями. Тогда же, в 1975 году, Bell Labs выпускает шестую редакцию системы, а в 1976 году выходит книга Джона Лайонса «Комментарии к 6-й версии UNIX с исходным кодом», содержащая текст исходного кода ядра шестой версии AT&T UNIX и комментарии к исходным текстам, которые объясняли функционирование операционной системы UNIX. Последней версией UNIX была седьмая — затем наступила эра множества версий UNIX от разных производителей. В седьмой версии появился интерпретатор командной строки Bourne shell (заново переписанной версией которого является современный bash). Он интересен также тем, что был создан под влиянием языка Алгол-68 — потомка языка Алгол и непосредственного конкурента другого дочернего языка, Паскаля.

Таким образом, именно на третье поколение пришлось по-настоящему революционные события в развитии вычислительной техники. Многие из разработанного в тот период в Unix применяются до сих пор.

Следующий этап развития компьютера — четвертое поколение, связанное с еще большей интеграцией устройства и возможностью реализации уже внутри одной микросхемы целого ряда блоков, до этого реализуемых независимо друг от друга. Многие связывают четвертое поколение с созданием компанией Intel микропроцессора, реализующего на одной схеме регистры, умножители, арифметико-логическое и управляющее устройство.

Процессор

Рассмотрим историю процессоров (в терминологии периода их появления — микропроцессоров), сейчас широко распространенных и ставших стандартом. Речь идет об Intel-совместимых процессорах.

История Intel-процессоров

Исторически первым процессором, предшественником современных Intel-совместимых и некоторых других процессоров, был 4004. Он был разработан компанией Intel в 1971 году по заказу японской организации Nippon Calculating Machine, Ltd., впоследствии переименованной в Busicom, и предназначался для производства печатающих калькуляторов (UNICOM 141-P / BUSICOM 141-PF). Заказчики хотели получить в итоге калькулятор на 12 микросхемах — при этом планировалось прошивкой ПЗУ вносить изменения вместо разработки новых микросхем. Это была отличная находка, но еще более удачным стало предложение инженера Intel Теда Хоффа сократить число микросхем до четырех, реализовав арифметико-логическое устройство, регистры, управляющее устройство и т. д. в одной микросхеме.

В составе калькулятора использовались следующие микросхемы:

- 4001 — 256 байтовое ПЗУ;
- 4002 — 40-байтовое ОЗУ;
- 4003 — 10-битный расширитель ввода-вывода (сдвиговый регистр, превращающий последовательный код в параллельный для связи с клавиатурой и принтером);
- 4004 — процессор.

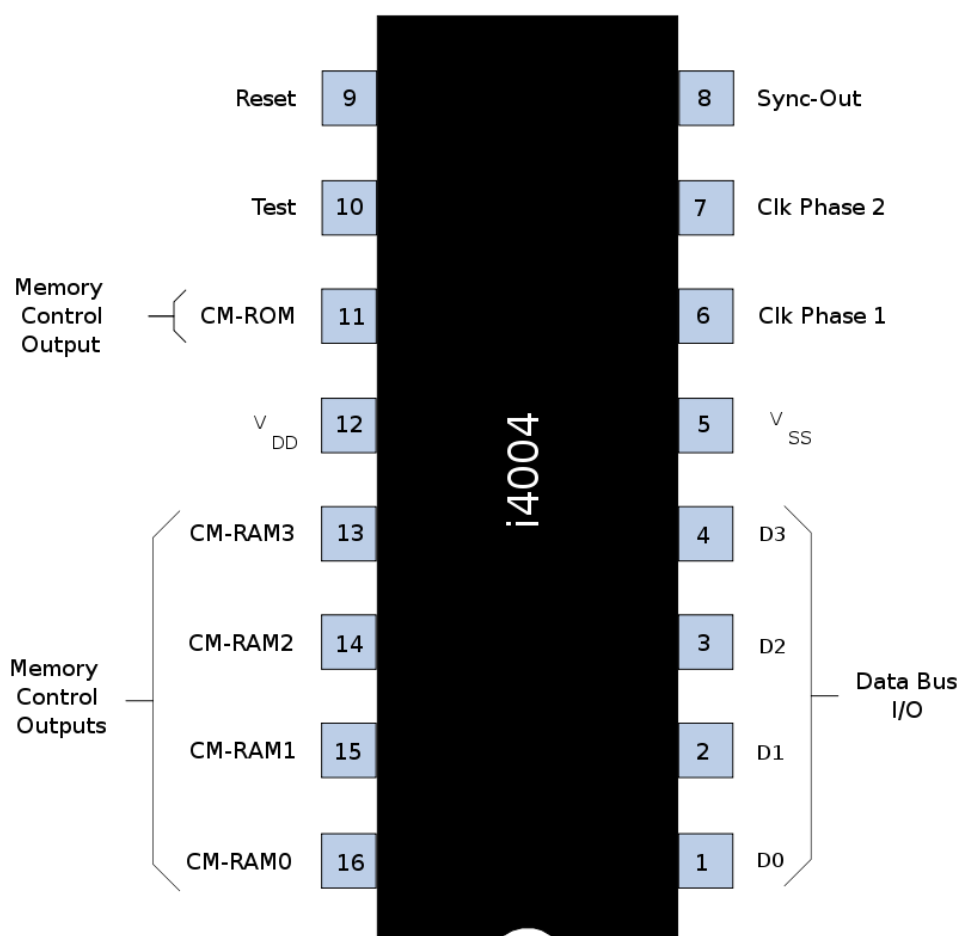
Откуда такая нумерация? Изначально Intel производили ПЗУ, которые имели нумерацию вида 1xxx. Соответственно, ОЗУ и сдвиговый регистр относились к сериям 2xxx и 3xxx, а процессорам было присвоено наименование вида 4xxx. Так как остальные три устройства предназначались для работы с процессором 4004, они тоже получили альтернативную нумерацию вида 400x, указывающую на работу с процессором. Однако процессор был разработан позже, и поэтому четвертый знак он получил больший, чем разработанные ранее ПЗУ и ОЗУ.

Конечно, операционной системы в привычном понимании у калькулятора не было, но была прошивка, которая позволяла ему выполнять ее функции (с восстановленной прошивкой можно ознакомиться по адресу http://www.4004.com/assets/Busicom-141PF-Calculator_asm_rel-1-0-0.txt).

Подход, при котором контроллер снабжается прошивкой — одной выполняющейся программой, позволяющей устройству реализовывать свои функции, — применяется до сих пор. Но подобно тому, как 4004 эволюционировал до современных процессоров, многие нынешние устройства также содержат в прошивке уже не одну программу, а целые ОС: в частности, есть и устройства, использующие прошивку на базе ядра Linux (и даже ОС GNU/Linux).

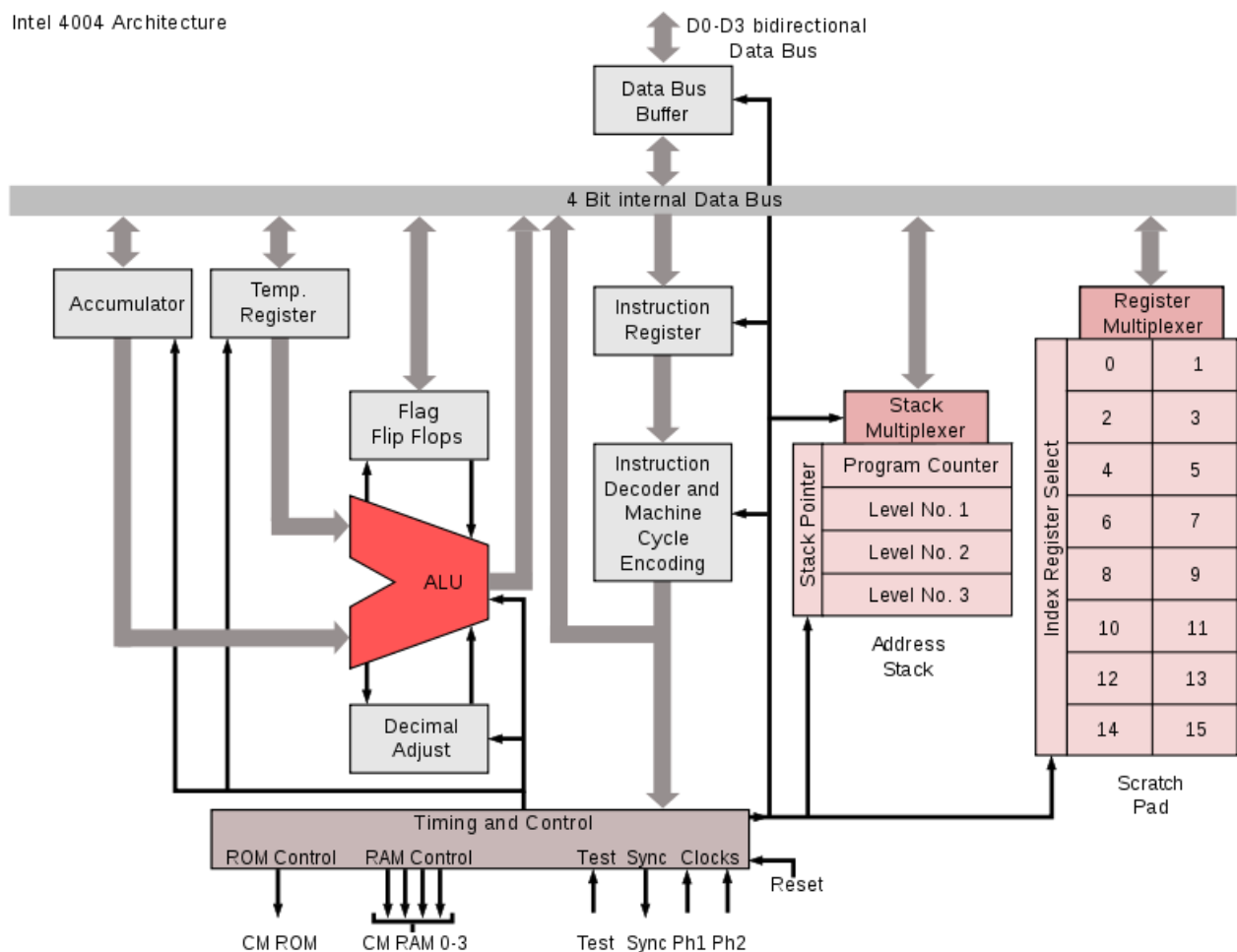


Калькулятор BASICOM 141-PF



Контакты микросхемы 4004

Intel 4004 Architecture



Блок-схема процессора 4004

Автор: Appaloosa - собственная работа, CC BY-SA 3.0, <https://commons.wikimedia.org/w/index.php?curid=2954987>

Следующий процессор 4040 был улучшенной версией процессора 4004 и вместо 16 включал уже 24 вывода. Кроме того, в отличие от предшественника, в процессоре 4040 появилась поддержка прерываний, а у микросхемы — новые входы. С datasheet для процессора 4040 можно ознакомиться по адресу <http://www.edgar-elsen.de/4040.pdf>

Следующий процессор, выпущенный в 1972 году, был уже 8-битным. Его создатели продолжали придерживаться схемы присвоения наименований по тому же принципу, и новый процессор получил название 8008.

История этого процессора в чем-то повторяет историю 4004. В 1969 году компания Computer Terminal Corporation, в дальнейшем переименованная в Datapoint, заказывает у Intel процессор, который планировалось использовать в новом терминале Datapoint 2000. Как и Busicom, Datapoint планировали разместить систему на нескольких микросхемах, но инженер Intel Тед Хофф предложил разместить все компоненты нового процессора на одной микросхеме. Когда она была готова, Datapoint разорвали контракт, но у процессора нашлись новые заказчики. Примечательно, что на базе микропроцессора 8008 в 1974 году были разработаны мини-компьютеры Mark-8 и Scelbi-8N.

Следующим шагом стало появление 8-битного процессора 8080. Хотя он был 8-разрядным и содержал семь 8-битных регистров (A, B, C, D, E, H, L), у него были ограниченные возможности обработки 16-разрядных чисел, для чего регистры объединялись в пары: BC, DE, HL.

Процессор 8080 оставил не меньший след в истории, чем 4004. На его базе MITS разработали компьютер «Альтаир-8800» — первый компьютер, который можно было собрать дома.



Альтаир-8800

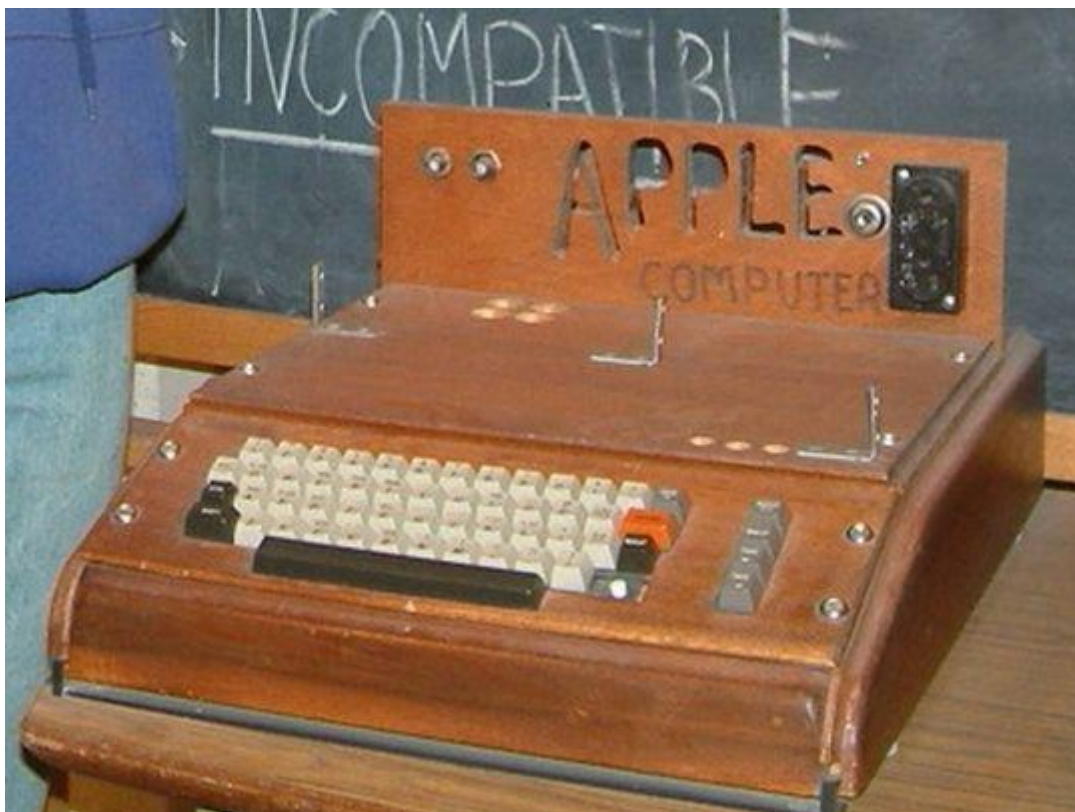
Микрокомпьютеры по возможностям и вычислительным мощностям уступали старшими братьями третьего поколения (впрочем, со временем это упущение было исправлено) — вследствие дешевизны и массовости производства машины. ЭВМ не имела ни клавиатуры, ни экрана, а программы необходимо было вводить в двоичной форме, переключая набором маленьких ключей, которые могли занимать два положения — вверх и вниз; результаты считывались также в двоичных кодах лампочками-индикаторами. Операционная система у таких устройств отсутствовала, взаимодействие оставалось на уровне машин первого поколения, хотя по сравнению с первыми ЭВМ это была более компактная и надежная вычислительная машина.

С «Альтаир-8800» начался путь компании Microsoft (тогда еще Micro soft). Побеседовав с Генри Робертсом, разработчиком компьютера, Билл Гейтс и Пол Аллен договорились о новой встрече через пару недель, чтобы рассмотреть интерпретатор языка Basic для «Альтаира». На самом деле у Гейтса и Аллена не было не то что интерпретатора, но и самого «Альтаира». Для разработки интерпретатора Аллен, пользуясь документацией, написал эмулятор «Альтаира» на PDP-10. Так Altair Basic стал первым языком программирования, написанным для микрокомпьютеров, и одновременно операционной системой для «Альтаир-8800». Дело в том, что интерпретируемый язык может выступать в роли операционной системы. Более того, для многих микрокомпьютеров в дальнейшем та или иная вариация языка Basic являлась операционной системой, так как была прошита в ПЗУ и загружалась при старте. Современные ОС также обладают интерфейсом командной строки, предоставляющей, по сути, возможности языка программирования — как, например, bash.

Altair Basic был основным источником дохода Microsoft до разработки MS DOS.

Конец 70-х годов ознаменовался расцветом 8-битных процессоров. Это были как совместимые аналоги от компаний NEC, Siemens, Zilog, AMD, так и конкуренты:

- Motorola 6800, которая нашла применение в машине MITS Altair 680 — с заменой i8080 на Motorola 6800; и ее аналог и конкурент от MOS Technology — 6502, использовавшийся в платформах NES, известной в России под именем Dendy;
- Atari 800, Atari 2600, Commodore 64, Apple I и II, копии Apple — болгарский «Правец» и советский «Арат».



Apple I

Автомат: Photo taken by rebelpilot - rebelpilot's Flickr Site, CC BY-SA 2.0, <https://commons.wikimedia.org/w/index.php?curid=183820>

Отдельно стоит упомянуть процессор Z80 от компании Zilog. Этот восьмибитный процессор разработали в 1978 году инженеры, непосредственно участвовавшие в создании процессора 4004. За основу был взят процессор 8080, и Z80 был с ним двоично совместим, что позволяло выполнять на процессорах Z80 программы, написанные для 8080, без изменений (среди них была и дисковая операционная система CP/M). При этом в процессоре сделали ряд улучшений: увеличили число регистров (что в принципе позволяло реализовывать микроконтроллеры без ОЗУ), установили один источник питания с напряжением +5В вместо трех (+5В, -5В и +12В) у Intel 8080. На базе Z80 производились известные микрокомпьютеры («бытовые компьютеры»), подключаемые к телевизору и магнитофону в качестве запоминающего устройства — например, ZX80, ZX81 и по-настоящему известный ZX Spectrum. Примечательно, что, несмотря на поддержку процессором Z80 прерываний, эти машины прерывания не использовали.

У машин Z80 встроенной в ПЗУ системой был язык программирования Sinclair Basic — это диалект Бейсика с однобайтовыми командами (при хранении, но не при печати на экран). В составе команд были инструкции:

- LOAD и SAVE для загрузки и сохранения программ с магнитофона;
- RUN — для запуска (многие программы в машинных кодах изначально содержали загрузчик на Sinclair Basic);
- PEEK и POKE — непосредственно для чтения и записи ячейки памяти;
- IN и OUT — для доступа к портам.

Для запуска программы в машинных кодах использовалась функция USR, которая возвращала значение регистра BC. Таким образом загружались программы в машинных кодах (что можно сравнить с COM-файлами в дисковых ОС), и так же можно было использовать встроенные функции ОС, вызывая их по прямому адресу. Привычного механизма вызова функции через прерывания устройство не имело. Кроме того, ассемблер Z80 отличался от Intel-ассемблера: например, мнемоникой LD вместо MOV.

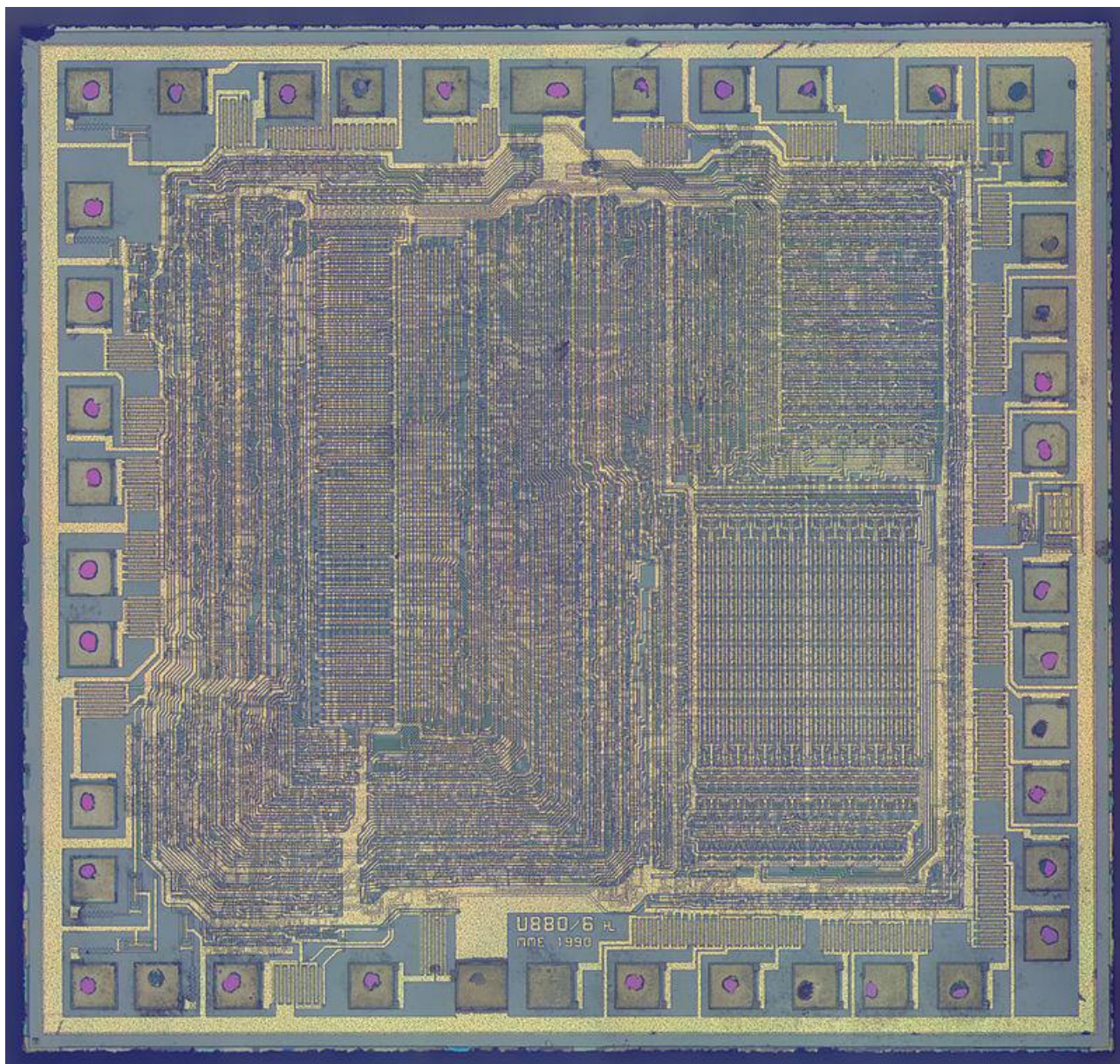
Подробнее с работой устройства можно ознакомиться в [этом материале](#).



ZX Spectrum

Автор: Bill Bertram — собственная работа, CC BY-SA 2.5, <https://commons.wikimedia.org/w/index.php?curid=170050>

Клоны процессоров 8080 и Z80 активно создавались в советское время. На базе микросхемы КР580ВМ80А (клона 8080) собирались любительские компьютеры «Микро-80», «Радио-86РК», «Микроша», использовавшие телевизор и бытовой магнитофон, а также ZX-Spectrum. Схемы для любительской сборки распространялись через журнал «Радио». В институте ядерной физики МГУ был разработан компьютер «Корвет», получивший распространение в школах, — известен образовательный класс КУВТ «Корвет». «Корвет» содержал в ПЗУ Basic, но позволял загружать с диска операционные системы CP/M и «Микродос» (советская ОС на основе CP/M). Интересной особенностью «Радио-86РК» было то, что в системе совсем не использовались прерывания, а управление звуком осуществлялось как раз выводом прерывания через переключение машинных команд EI («включить обработку прерываний») и DI («отключить обработку прерываний»). На базе КР1858ВМ1 (клона ZX80) создавались многочисленные советские клоны ZX-Spectrum, а также телефоны с АОН — автоматическим определителем номера.



Микросхема KP1858BM1

Автом: ZeptoBars — <http://zeptobars.ru/en/read/KR1858VM1-Z80-MME-Angstrom>, CC BY 3.0, <https://commons.wikimedia.org/w/index.php?curid=33458670>

Следующим процессором от Intel был 8086. В отличие от своих предшественников, он был уже 16-битным. Любопытно, что и современные процессоры могут выполнять инструкции, предназначенные для 8086.

Процессор 8086 содержал четырнадцать 16-разрядных регистров:

- 4 регистра общего назначения (AX, BX, CX, DX);
- 2 индексных регистра (SI, DI);
- 2 указательных регистра (BP, SP);
- 4 сегментных регистра (CS, SS, DS, ES).

Для нужд процессора использовались указатель команды (IP) — фактически программный счетчик — и регистр флагов FLAGS, состоявший из 9 битовых флагов, которые можно было воспринимать как битовые регистры и управлять ими особым образом. При этом регистры данных (AX, BX, CX, DX) допускали адресацию не только целых регистров, но и их младшей половины (регистры AL, BL, CL, DL) и старшей половины (регистры AH, BH, CH, DH), что позволяло работать с ними как с 8-битными

регистрами. Последнее давало возможность выполнять на процессоре не только новое 16-разрядное программное обеспечение, но и сохранять совместимость с 8-битными программами, хотя непосредственный перенос был невозможен. Аналогичные команды процессора имели другие байтовые последовательности команд. Так, в 8080 и ZX80 команда NOP («ничего не делать»), служившая для заполнения и выравнивания адресов, имела код 0×00 , а в 8086 и далее — 0×90 , а 0×00 стала составной частью одной из команд сложения. Таким образом, несмотря на совместимость на уровне мнемоник, для переноса программу пришлось бы перекомпилировать.

16-разрядный регистр позволял распоряжаться $2^{16}=65536$ байт памяти, то есть порядка 64Кб, несмотря на то, что системная шина позволяла адресовать до 1 Мб памяти. Для этого была разработана сегментная адресация памяти с использованием до четырех сегментов по 64 Кб.

В процессоре 8086 был реализован прототип конвейерной обработки: в то время, когда процессор выполнял одну инструкцию, уже происходило чтение следующей команды.

Процессор 8086 применялся в первом переносном компьютере — Xerox Note Taker, предшественнике ноутбуков. Он использовал целых три процессора 8086: в качестве основного ЦПУ, в качестве графического процессора и для управления вводом-выводом.



Xerox Note Taker

Модификация 8088 применялась в революционной серии компьютеров IBM PC/XT. В погоне за сохранением рынка компания IBM решила использовать в новой ЭВМ узлы других производителей, фактически создав конструктор, что предопределило дальнейшую судьбу персональных компьютеров.

Отечественные клоны 8086 и 8088 — К1810ВМ86 и К1810ВМ88 — использовались в советских машинах ЕС ПЭВМ начиная с ЕС-1840. Машины были IBM-совместимыми.

Шагом в дальнейшем развитии машин стали 16-битные процессоры 80186 и 80286. Если первый практически не применялся в ЭВМ, то на базе второго появилось новое поколение компьютеров IBM AT. В отличие от 8086, в 80286 адреса и данные передавались через разные контакты процессора, что позволяло устройству действовать быстрее. В нем появился защищенный режим памяти и новые регистры для его обслуживания. 80386 уже был 32-байтным процессором, в котором появились 32-битные регистры (EAX, EBX), страничная организация памяти, аппаратная поддержка многозадачности.

Разрядность процессора

В предыдущем тексте упоминаются 4-разрядные, 8-разрядные, 16-разрядные процессоры. Разберемся, что это означает. Разрядность процессора — это показатель числа бит, с которыми процессор может манипулировать одновременно, то есть размер машинного слова.

Что означает понятие «4-битный процессор»? Число, с которым может он работать, лежит в диапазоне от 0000 до 1111, то есть от 0 до 15 — ровно 16 значений (2^4). Возможна ли работа с большими числами? Да, но для этого потребуется большее число инструкций процессора.

Стоит увеличить число бит в два раза, и мы получим уже 256 значений (2^8)! Такое машинное слово называется байт, и может содержать значения от 0000 0000 до 1111 1111 (от 0 до 255).

Таким образом, 8-битный процессор уже мог работать с символами ASCII, которые содержат ровно 256 знаков и управляющих конструкций.

16-битный процессор (2^{16}) — с 65 536 значений. Он оперирует 16-битными словами, состоящими из двух байт.

Для 32-битного и 64-битного значения несложно рассчитать самостоятельно. 32-битная система оперирует 32-битными словами, состоящими из 4 байт, 64-битная система — уже состоящими из 8 байт.

Количество адресуемой памяти зависит от разрядности процессора. Адрес ячейки памяти задается одним или парой регистров. Технически возможно реализовать, например, 8-битный доступ к памяти на 4-битном компьютере с использованием комбинации двух регистров и других решений. Тем не менее известно, что 32-битные компьютеры, как правило, адресуют до 4-х гигабайт памяти. Почему? $2^{32}=4$ Гб. Поэтому 64-битные процессоры могут использовать оперативную память, превышающую 4 Гб.

Регистры процессора

В состав процессора входит сверхбыстрая оперативная память (СОЗУ), которая располагает именованными ячейками. Работа с регистрами — наиболее быстрая, так как происходит внутри ЦПУ и не требует обращения к внешним по отношению к нему устройствам. СОЗУ более дорогая и, соответственно, ограниченная, потому в СОЗУ может размещаться лишь небольшой объем данных. Как правило, это данные, необходимые для обеспечения работы процессора, и промежуточные данные вычислений (как арифметических, так и предназначенных для передачи номеров функций и аргументов при вызове программных прерываний).

Регистры могут быть сгруппированы таким образом, что к двум можно обращаться как к одному. Рассмотрим это на примере регистра А (аккумулятора).

Процессор 8080 (результат дальнейшего развития восьмибитного 8008, также 8-битный) насчитывал регистры А, В, С, D, Е, H, L, но имел ограниченные возможности обработки 16-разрядных чисел, для чего регистры объединялись в пары BC, DE, HL.

В составе процессора есть основные регистры, без которых невозможно выполнение базовых задач процессора, и дополнительные (например, регистры сопроцессора) для выполнения специфических задач. Регистр имеет разрядность, которая характеризует, сколько байт регистр может хранить. Разрядность процессора, как правило, описывает разрядность его основных регистров. Но в процессоре могут присутствовать регистры как меньшей разрядности (например, для совместимости), так и большей (для сопроцессора). Кроме того, разрядность не означает, что в принципе нельзя оперировать с числами большей разрядности. Это возможно либо с выполнением операций за несколько тактов (за один такт выполняется одна машинная инструкция) и сохранением промежуточных

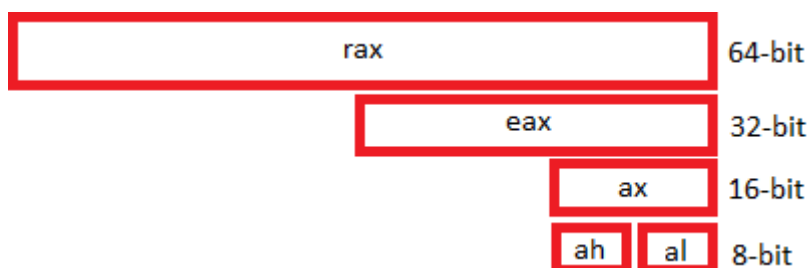
результатов, либо с применением дополнительных средств (сопроцессоров, группировки нескольких регистров для хранения частей одного числа).

В 16-битном процессоре 8086 уже имелся 16-битный регистр AX, который можно было рассматривать как пару восьмибитных регистров AH и AL (high и low).

В 32-битном процессоре 80386 появились 32-битные регистры, к младшей части которых можно было обращаться по-прежнему. Так, если аккумулятор в 32-битной версии имел название EAX, то к его младшей 16-битной части можно было обратиться как к AX, а к ее половинкам — как к AH и AL. Старшая часть регистра такого расположения не получила — отдельный доступ имелся только к младшей части.

Аналогично сложилась история и с 64-битными процессорами. 64-битный регистр аккумулятора имеет наименование RAX, младшая 32-битная часть которого по-прежнему доступна как EAX. В свою очередь, ее младшая 16-битная часть доступна как AX, а обе ее части — как AH и AL.

Все вместе это выглядит следующим образом:



Обратите внимание: изменяя, например, регистр AL, вы изменяете и AX, и EAX, и RAX (при этом AH не изменится). Изменяя AX, с большой долей вероятности вы поменяете и AH, и AL (это зависит от того, совпадает ли старое и новое число в старшей и младшей своей части или нет). Таким образом, несмотря на возможность работы с отдельными «частями» регистра, вы работаете со всем регистром в целом! При этом иногда бывает удобно работать, например, с AH и AL как с отдельными регистрами, не воспринимая их как единое целое.

Какие бывают регистры

Что такое регистры? На высоком уровне их можно воспринимать как особые переменные. Более того, если написать такую строчку:

```
for (i; i++; i<100) { s=s+2; }
```

... то при компиляции программы роль переменной *i* будет играть регистр CX. Кстати, и в качестве переменной *S* также может выступить регистр AX — впрочем, не везде и не всегда, так как программа на высоком уровне будет выглядеть сложно после компиляции. Одна и та же переменная может находиться в разных участках памяти и выполнять функцию регистра.

Регистры общего назначения

Содержат 4 регистра счетчиков-аккумуляторов и 4 регистра для работы с памятью (кроме того, для работы программы с памятью, есть еще внутренние регистры, напрямую недоступные для изменения программой).

Регистры счетчиков-аккумуляторов:

- **RAX/EAX/AX: Accumulator register** — аккумулятор, применяется для хранения результатов промежуточных вычислений;

- **RBX/EBX/BX: Base register** — базовый регистр, применяется для хранения адреса (указателя на объект в памяти);
- **RCX/ECX/CX: Counter register** — счетчик, его неявно используют некоторые команды для организации циклов (см. loop);
- **RDX/EDX/DX: Data register** — регистр данных, используется для хранения результатов промежуточных вычислений и ввода-вывода.

Нижняя 16-битная часть каждого из вышеперечисленных регистров также состоит из половинок (AH:AL, BH:BL, CH:CL, DH:DL).

Следующие четыре регистра, как правило, не делятся на 8-битные половинки:

- **RSP/ESP/SP: Stack pointer register** — указатель стека, содержит адрес вершины стека;
- **RBP/EBP/BP: Base pointer register** — указатель базы кадра стека (stack frame), предназначен для организации произвольного доступа к данным внутри стека;
- **RSI/ESI/SI: Source index register** — индекс источника, в цепочечных операциях содержит указатель на текущий элемент-источник;
- **RDI/EDI/DI: Destination index register** — индекс приемника, в цепочечных операциях содержит указатель на текущий элемент-приемник.

Сегментные регистры

- **CS: Code segment** — содержит текущий сегмент кода. Процессор может загружать инструкции только из указанного в CS сегмента. Регистр CS используется процессором в связке CS:IP. Не может меняться непосредственно, командой MOV. Тем не менее процессорные команды вызовов и переходов (JMP, CALL, RET) фактически могут изменять этот регистр;
- **DS: Data segment** — содержит текущий сегмент данных. Может использоваться в комбинации DS:BX, DS:SI, DS:DI (в формате сегмент:смещение) для манипуляции с данными;
- **SS: Stack segment** — содержит текущий сегмент стека. Фактически регистры SP и BP указывают смещение относительно сегмента, заданного в SS (то есть работа идет с памятью в SS:SP и SS:BP);
- **ES: Extra segment** — дополнительный сегмент, часто используется неявно в строковых командах как сегмент-получатель (работа идет с ES:DI);
- **FS: F segment** — дополнительный сегментный регистр без специального назначения;
- **GS: G segment** — дополнительный сегментный регистр без специального назначения.

В ОС Linux используется плоская модель памяти (flat memory model), в которой все сегменты описаны как использующие все адресное пространство процессора и, как правило, явно не применяются. Таким образом, каждый адрес представляется в виде 32-битных смещений. Это позволяет одним 32-битным регистром адресовать до 4 Гб памяти и упрощает программирование. Тем не менее работать с сегментами можно.

Обратите внимание: сегментные регистры — 16-битные. Почему? Ответьте на этот вопрос самостоятельно.

Регистры, связанные с состоянием процессора

- **RIP/EIP/IP — instruction pointer.** Данный регистр указывает на адрес текущей команды и не может напрямую изменяться командами **MOV**, **ADD** и т. д. Выполнение каждой команды либо

изменяет этот регистр на размерность команды, либо меняет на указанный адрес. Фактически команды **JMP**, **RET**, **REP**, **LOOP**, **CALL** изменяют состояние данного регистра. Команду **NOP**, которая ничего не делает, можно считать командой **INC IP**. Команду **JMP XX** можно изобразить как **MOV IP, XX** (обычно так не делают, хотя сам процессор именно так воспринимает эти команды);

- **RFLAGS / EFLAGS / FLAGS** — регистр флагов. Если AX можно воспринимать как два 8-битных регистра AH и AL, чтобы работать с ними напрямую, регистр флагов состоит из однокбитных регистров. Работа с ними напрямую невозможна, но ряд команд по-разному выполняются в зависимости от значения данных флагов.

Некоторые значимые флаги

ZF: zero flag — флаг нуля:

- 1 — результат последней операции нулевой;
- 0 — результат последней операции ненулевой.

IF: interrupt flag — флаг прерываний:

- 1 — в этом случае процессор обрабатывает внешние (аппаратные прерывания);
- 0 — в этом случае процессор игнорирует внешние прерывания (кроме NMI).

CF: carry flag — флаг переноса:

- 1 — во время арифметической операции был произведен перенос из старшего бита результата;
- 0 — переноса не было.

OF: overflow flag — флаг переполнения:

- 1 — во время арифметической операции произошел перенос в / из старшего (знакового) бита результата;
- 0 — переноса не было.

DF: direction flag — флаг направления. Указывает направление просмотра в строковых операциях:

- 1 — направление «назад», от старших адресов к младшим;
- 0 — направление «вперед», от младших адресов к старшим.

Пример использования:

```
if (a==7)
{ // обработка кода 7
}
else
{
    // иначе
}
```

На ассемблере это будет выглядеть так:

```
CMP AX, 0x07 ; сравниваем регистр AX и число 7

JZ xx ; если ZF=00 переходим к xx — обработка кода 7

здесь иначе.

xx: обработка кода 7
```

Есть еще вариант записи:

```
CMP AX, 0x07

JE xx
```

Эта запись передает то же самое (синоним), но в ней мы ориентируемся уже не на флаг нуля, а на равенство, то есть результат **Compare — Equal**. Точно так же дело обстоит с переходом в случае ненулевого результата: **JNZ** — если **ZF!=0** или, что то же самое, **JNE** — результат **Compare — Not Equal**.

Несмотря на то, что у каждого флага свои механизмы работы (например, для установки и сброса **IF** есть команды **STI** и **CLI**), существует общий способ работы с регистром флагов: **PUSHF** запишет значение регистра **FLAGS** на вершину стека, а **POPF** — извлечет значение с вершины стека и запишет в регистр **FLAGS**.

Порядок байтов

Существует как минимум два способа хранения и передачи байтов, которые могут различаться в разных реализациях процессоров: **big-endian** и **little-endian**. Названия этих способов взяты из книги Дж. Свифта «Путешествие Гулливера», где упоминаются жители страны, воевавшие между собой из-за того, как правильно разбивать вареное яйцо — с тупого конца (**big-endians**, тупоконечники) или с острого (**little-endians**, остроконечники). (Кстати, на похожую тему существует мультфильм [«Хроники Бутербродной войны»](#)).

Big-endian («тупоконечный», от старшего к младшему) — это интуитивно понятный, привычный порядок байтов. Он также называется сетевым порядком байтов (**network byte order**) и используется в процессорах **SPARC**, **MIPS***. Пример: число 539 в шестнадцатеричной системе счисления для — **02 1b**.

Little-endian («остроконечный», от младшего к старшему) — это противоположность вышеописанному порядку байтов. Он более удобен для арифметических операций, но хуже считывается при чтении или

правке машинного кода. Little-endian также называется интеловским порядком байт (**intel byte order**) и применяется в процессорах Intel. Пример: число 539 в шестнадцатеричной системе счисления для 16-битной машины в привычном нам Big-endian представлении (старшие слева) будет выглядеть как 021B. Калькуляторы также работают с таким представлением. Но в памяти и в регистре процессора, работающего с Little-Endian порядком байт (например, на Intel-процессорах), данное число будет храниться в виде 1B02.

Если же мы возьмем 32-битный процессор, то это число будет уже храниться в Big-Endian как 0000021B, а в Little-Endian как 1B020000. Но обратите внимание, что внутри байта порядок бит не меняется.

	Шестнадцатеричная запись	Двоичная запись
Big-Endian	00 00 02 1B	0000 0000 0010 1011
Little-Endian	02 1B 00 00	1011 0010 0000 0000

В 64-битном процессоре данные числа будут представлены уже в таком виде:

Big-Endian: 00 00 00 00 00 00 02 1B

Little-Endian: 1B 02 00 00 00 00 00 00

Существуют линейки процессоров с поддержкой обоих типов либо с возможностью переключения переключкой на плате. К ним относятся **ARM** и **MIPS*** (мы сталкивались с реализацией **MIPS** Big-endian, но в общем случае они могут быть как big-endian, так и little-endian).

Для преобразования одного порядка байтов в другой существуют разные механизмы. Так, в C для этого служат функции **ntohs** (network to host short), **htons** (host to network short), **ntohl** (network to host long), **htonl** (host to network long), которые производят соответствующее преобразование, если порядок байтов отличается сетевого, или ничего не делают, если порядок совпадает.

В ассемблере 32-битных процессоров Intel имеется инструкция **bswap** (например, `bswap eax`), которая преобразует порядок байт регистра в противоположный. Ее аналог для 16-битных процессоров — команда **xchg** (например, `xchg al,ah`), которая меняет два регистра местами. Почему именно эта команда преобразует порядок байтов, вам предлагается ответить самостоятельно исходя из информации о регистрах Intel-совместимых процессоров.

В файлах Unicode-16 в начале файла может использоваться **Byte order mark (BOM)** — двухбайтовый символ, который имеет значение **0 × FEFF**. Символа **0 × FEFF** не существует, поэтому если файл начинается с **0 × FEFF**, это знак, что файл необходимо преобразовать из little-endian в big-endian.

Прерывания

Прерывание — это функция, имеющая номер, по которому производится ее вызов. Отличие такой функции от обычных состоит в том, что вызов обычной функции осуществляется по адресу (например, **CALL 0 × 0110**), а вызов прерывания — по номеру (например, **INT 3**). Еще одно отличие в том, что работа с прерываниями — одна из важнейших особенностей современных процессоров, и в ряде случаев не программы, а сам процессор по определенным сигналам способен вызывать прерывания, с последующим возвратом к следующей инструкции, на которой завершилось выполнение.

Из самого названия «прерывание» следует, что оно останавливает работу программы и процессор переходит к обработчику прерывания. Пример с **INT 3** — это случай, когда прерывание явно вызывается из программы. Но программа может быть остановлена, и процессор переходит к обработчику прерывания и без соответствующей инструкции — например, при наступлении какого-либо события.

Операционная система включает таблицу векторов прерываний, сопоставляя номера прерываний и фактические адреса нахождения обработчиков прерываний. Обработчики прерываний могут перекрываться другими, сохраняя адрес старого прерывания и вызывая его в процессе выполнения собственного кода.

Исключения

Исключения — это прерывания, которые возникают в случае событий, связанных с выполнением программы (деление на ноль, ошибка сегментации, переполнение стека и т. д.) В таких случаях программа останавливается и управление передается обработчику. Как правило, программа может переопределить прерывание и продолжить работу, но обычно происходит снятие программы с выполнения и вывод соответствующей ошибки. Например, прерывание по делению на ноль вызовет инструкция **DIV**.

При обработке прерывания регистры **IP** и **FLAGS** сохраняются в стеке, после чего управление переходит к обработчику прерывания. После завершения прерывания **IRET**, если управление будет возвращено назад, **IP** и **FLAGS** восстанавливаются — фактически восстановление **IP** и есть переход к возобновлению инструкции.

Аппаратные прерывания

Аппаратные прерывания обрабатываются при поступлении сигнала **IRQ** на вход процессора. Они служат для взаимодействия процессора с устройствами: например, нажатие кнопки на клавиатуре приводит к поступлению соответствующего прерывания на процессор, и обработчик прерывания считывает код введенного символа, который затем может быть запрошен соответствующим вызовом операционной системы. Номера **IRQ** в настоящее время не совпадают с номерами **INT**.

Для управления обработкой прерываний служит флаг **FI**. Инструкции установки флага соответствует флаг **STI** (Set Interrupt-Enable Flag), инструкции сброса флага — **CLI** (Clear Interrupt-Enable Flag). Если флаг сброшен, внешние прерывания, за исключением немаскируемого (**NMI**), игнорируются. Примечательно, что команда **HLT** служит для остановки процессора до поступления внешнего прерывания. Комбинация **CLI HLT**, таким образом, полностью останавливает процессор.

Немаскируемое прерывание

INT 2 — немаскируемое прерывание **NMI** — поступает на вход процессора **NMI**, и в этом случае его обработчик будет выполнен, даже если **IF** сброшен. Название данной операции означает, что прерывание не может маскироваться сбросом флага **IF**. **NMI** генерируется, например, при ошибке четности памяти. Память в ОЗУ хранится в банках по 9 бит, девятый бит — контроль четности, который позволяет на аппаратном уровне отслеживать корректность работы ОЗУ и используется, чтобы количество единиц в 8 битах было нечетным. Сбой четности приводит к вызову прерывания **NMI** и, как правило, к остановке машины. В других процессорах и микроконтроллерах **NMI** может служить для других целей: для удаленного управления процессором, вывода его из **HLT** по событию и т. д. Интересно, что первые ОЗУ давали сбой из-за альфа-частиц, образующихся при распаде примесей урана или тория, который мог присутствовать в устройстве из-за технического несовершенства микросхем.

Прерывание для отладки — breakpoint

Большинство прерываний кодируются двумя байтами. Прерывание **INT 3** кодируется одним байтом **0 × CC**. Это специально вставленная точка останова, которая вызывает прерывание-обработчик для отладки.

Программные прерывания

Программные прерывания вызываются инструкцией **INT** для доступа к функциям системы. Обработчики прерываний в разных устройствах предоставлялись BIOS, операционной системой (системные вызовы), драйверами. В реальном режиме процессора любая программа могла осуществлять обработку программных прерываний. В настоящее время это позволено делать только компонентам операционной системы.

Системные вызовы

Изначально для системных вызовов использовалась инструкция **INT**, однако в **i586** появилась инструкция **sysenter**, а в 64-битных системах — уже **syscall**. Системные вызовы представляют собой механизм, похожий на прерывания, но в нем есть и свои отличия.

Языки программирования низкого уровня

Машинный код

Приведем пример машинного кода для **x86**, для ОС MS DOS. Формат исполняемого файла — **COM** (простой файл без заголовков, код помещается в память начиная с адреса **0 × 100**):

BB 11 01 B9 0D 00 B4 0E 8A 07 43 CD 10 E2 F9 CD 20 48 65 6C 6C 6F 2C 20 57 6F 72 6C 64 21

В машинном коде нет таких привычных вещей, как перевод строки и отступы. Это двоичный файл. Команды в нем имеют переменную длину, и удаление любого символа приведет к тому, что код будет непредсказуемым.

В приведенном примере команды отделены друг от друга цветом. На самом деле никакого разделения нет: читая очередной байт, процессор в зависимости от содержания этого байта может прочитать еще несколько байт. В целом аргументы команды не отделены от команды.

Для удобства запишем код построчно и разберем его:

```
BB 11 01
B9 0D 00
B4 0E
8A 07
43
CD 10
E2 F9
CD 20
48 65 6C 6C 6F 2C 20 57 6F 72 6C 64 21
```

- **BB 11 01** — команда поместить число 0×0111 (шестнадцатеричное, little-endian) в регистр BX. Это смещение строки «Hello, world!»;
- **B9 0D 00** — команда поместить число $0 \times 00D$ (шестнадцатеричное, little-endian) в регистр CX. Это длина строки «Hello, world!»;
- **B4 0E** — поместить число $0 \times 0E$ в регистр AH. Это номер запроса, который мы будем отправлять операционной системе (печать на экран);
- **8A 07** — поместить в регистр AL значение из памяти, адрес которой находится в регистре BX. Это второй аргумент для системного вызова — адрес печатаемого символа. Каждый раз мы будем переходить сюда;
- **43** — увеличить BX на 1. Это действие — для следующей итерации. Таким образом мы переходим к следующему байту строки;
- **CD 10** — вызвать прерывание 0×10 . Это обращение к ОС с запросом функции. В качестве аргументов вызова фигурируют регистр AH, содержащий номер функции $0 \times 0E$ вызова 0×10 , и содержащийся в регистре AL адрес очередного байта, включающего в себя печатаемый символ. Таким образом, мы печатаем по одному символу в позицию, где находится курсор. На самом деле 0×10 — это даже не функция MS DOS, а функция, реализованная в BIOS и представляющая собой пусть и медленный, но все же доступ к памяти. Для доступа к многочисленным функциям MS DOS использовалось прерывание 0×21 ;
- **E2 F9** — если CX не равно нулю, перейти на 7 байт назад начиная с адреса следующей с E2 F9 команды. (F9 в десятичной системе счисления означает -7. Почему именно F9? Потому что речь идет об отрицательном числе в дополнительном коде размером в один байт. Тем самым команда LOOP поддерживает переходы в диапазоне от -128 до 127);
- **CD 20** — завершить выполнение программы, выгрузить ее из памяти и передать управление операционной системе.
- **48 65 6C 6C 6F 2C 20 57 6F 72 6C 64 21** — Hello, world!

Мнемоники, опкоды, дизассемблер

На самом деле команды не имеют определенных имен — у них есть только номера кодов. Изначально процесс программирования выглядел так: программисты смотрели по справочнику номер команды и вписывали по очереди байты. Процессор до сих пор исполняет команды именно таким образом, однако разработка изменилась за счет появления более высоких уровней абстракции (мнемоники, собственно ассемблеры, язык С, языки высокого уровня и т. д.)

Пока машинный код несложен, номера команд можно даже помнить наизусть:

- **CD 10** — вызвать прерывание 10;
- **43** — инкрементировать BX.

Впрочем, программистам все равно приходилось пользоваться справочниками. Это тоже создавало неудобства, записи программ даже на бумаге были громоздкими, потому возникли мнемоники.

Ведь:

- **INT 10** — нагляднее, чем CD 10;
- **INC BX** — нагляднее, чем 43.

Таким образом, в мнемониках код можно было бы записать так:

```
MOV BX, 0111
MOV CX, 000D
MOV AH, 0E
MOV AL, [BX]
INC BX
INT 10
LOOP -7
INT 20
Hello, world!
```

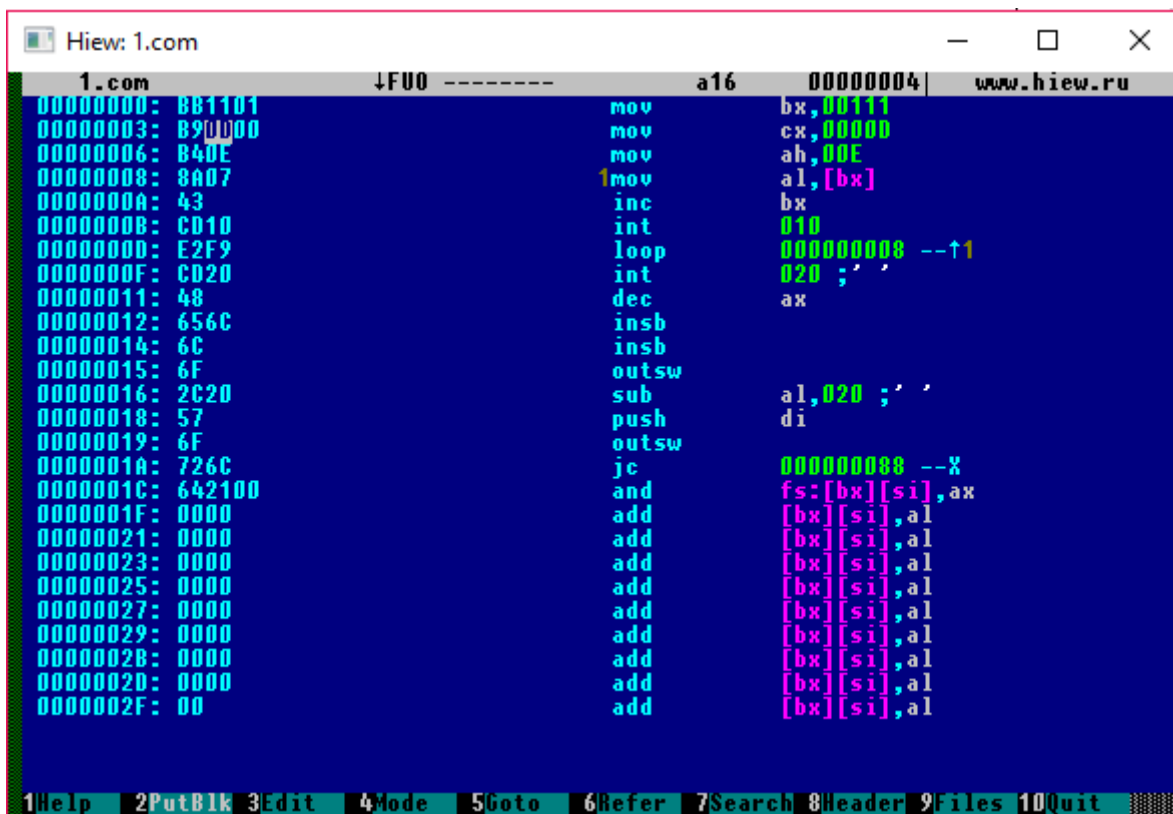
Такая запись получалась удобнее и производительнее, но переводить код все равно приходилось по таблицам, при этом точно рассчитывая позиции в байтах.

Следующим шагом к удобному кодированию стали программы «Автокод» и «Монитор». Они позволяли вводить код в мнемониках, что по существу было продолжением программирования в машинных кодах. Безусловно, удобнее было бы ввести метки, автоматический расчет адресов, куда необходимо сделать переход, добавить переменные, макросы и т. д. Такие программы появились, а сам язык, принадлежащий к более высокому уровню, чем машинный код или мнемоники, получил название Ассемблер.

Кроме того, существует и обратная ситуация, когда машинные коды необходимо перевести в мнемоники. Уровень подобного преобразования может быть разным — от простого проставления мнемоник до подсчета точек вхождения и даже именования вызовов. Такая процедура называется

дизассемблированием, близким к этому понятием является декомпиляция — перевод на более высокоуровневый язык.

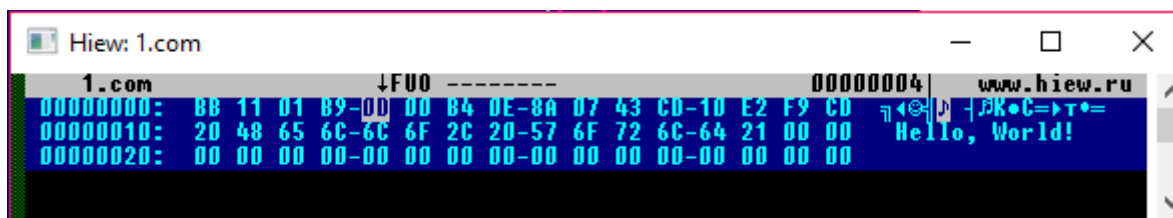
Рассмотрим дизассемблирование программы в Hiew.



```
1.com          ↓F00 -----          a16          00000004 | www.hiew.ru
00000000: B81101      mov     bx,0011
00000003: B90000      mov     cx,0000
00000006: B40E      mov     ah,00E
00000008: 8A07      1mov     al,[bx]
0000000A: 43      inc     bx
0000000B: CD10      int     010
0000000D: E2F9      loop    00000008 --↑1
0000000F: CD20      int     020 ; '
00000011: 48      dec     ax
00000012: 656C      insb
00000014: 6C      insb
00000015: 6F      outsw
00000016: 2C20      sub     al,020 ; '
00000018: 57      push    di
00000019: 6F      outsw
0000001A: 726C      jc      00000008 --X
0000001C: 642100     and     fs:[bx][si],ax
0000001F: 0000      add     [bx][si],al
00000021: 0000      add     [bx][si],al
00000023: 0000      add     [bx][si],al
00000025: 0000      add     [bx][si],al
00000027: 0000      add     [bx][si],al
00000029: 0000      add     [bx][si],al
0000002B: 0000      add     [bx][si],al
0000002D: 0000      add     [bx][si],al
0000002F: 00      add     [bx][si],al
```

Hiew — довольно интеллектуальная программа: обратите внимание на указание ссылки в LOOP. Тем не менее 32-битная версия Hiew уже не понимает формат COM и не опознает ссылку на «Hello, world!» (далее увидим, что Hiew умеет делать это для MZ). Зато адрес LOOP рассчитан верно, потому что задан относительно текущей инструкции (в дополненном коде это отрицательное число).

А вот то, что Hiew пытается дизассемблировать и «Hello, world!», получая непредсказуемый код, — нормально. Более того, в общем случае невозможно определить, данные это или код.



```
1.com          ↓F00 -----          00000004 | www.hiew.ru
00000000: B8 11 01 B9-00 00 B4 0E-8A 07 43 CD-10 E2 F9 CD 71 04 00 1B K=C=7*
00000010: 20 48 65 6C-6C 6F 2C 20-57 6F 72 6C-64 21 00 00 Hello, World!
00000020: 00 00 00 00-00 00 00 00-00 00 00 00-00 00 00 00 00
```

Код в шестнадцатеричном виде

Анализ машинного кода и внесение изменений в него применяются и сейчас — например, если в программе обнаружена ошибка, а исходных файлов нет. Иногда ошибки исправляют непосредственно в машинном коде — это тоже своего рода программирование в машинных кодах. Несмотря на то, что в дизассемблерах доступны мнемоники, код инструкции приходится вводить непосредственно. Очень часто происходят замены JZ на JNZ (чтобы исправить ошибку в ветвлении), исправление адресов в JMP, CALL, замена JMP на NOP. Команда NOP оказывается особенно полезной, так как просто удалить, например, ошибочный JMP не получится: смещение даже одного байта приведет в лучшем случае к изменению всех адресов (команды перехода не будут работать), а в худшем — к полной потере смысла в дальнейшем коде: с учетом переменной длины инструкций аргументы превратятся в совершенно

случайные команды, а код станет непредсказуемым. В этом случае NOP используется в качестве заполнителя.

Практическое задание

Ответьте на следующие вопросы в Google Docs, максимально подробно расписав, почему вы так считаете. Приложите ссылку на документ к уроку.

1. Что такое реле и в чем их ценность для IT-индустрии?
2. Какие плюсы у полупроводниковых транзисторов?
3. Почему изобретение микропроцессора было важным шагом?
4. Что такое регистры процессора?
5. Что такое прерывания?
6. Что такое машинный код?
7. Почему бинарный код проще читать в шестнадцатеричном формате?
8. Что такое порядок байт?
9. Чем определяется разрядность процессора?
10. Может ли процессор оперировать числами большей разрядности, чем разрядность процессора?

Задачи со * предназначены для продвинутых учеников.

Дополнительные материалы

1. [Деление на ноль на счетной машинке.](#)
2. [Down Yonder 88 note piano roll.](#)
3. [Hiew — wikipedia.](#)
4. [Справочник команд.](#)
5. [Anatomy of the EICAR Antivirus Test File.](#)

Используемая литература

Для подготовки данного методического пособия были использованы следующие ресурсы:

1. [Машинный код — Википедия.](#)
2. [Hiew — wikipedia.](#)
3. [Второе поколение ЭВМ 1959—1967.](#)
4. [Busicom-141PF-Calculator_asm_rel-1-0-0.](#)
5. [Разбираемся с прямым и обратным порядком байтов.](#)

6. [Multics — wikipedia.](#)
7. [Linux — syscalls. Системные вызовы в Linux.](#)
8. [Написание и отладка кода на ассемблере x86/x64 в Linux.](#)
9. [Defcon OpenCTF 2015 - Sigil.](#)